

EVA²Hub: Efficient Variable-Amount Anonymous Payment Channel Hub

Xiaofeng Jia, Pubin Jiang, Shuyu Zheng, Cong Zuo *Member, IEEE*, and Jun Shao, *Senior Member, IEEE*

Abstract—Off-chain payment channel hubs (PCHs) promise high-throughput cryptocurrency payments via untrusted intermediaries, but existing designs either leak payer–payee relationships or payment values, restrict payments to fixed denominations or one direction, or allow griefing and abort attacks that lock honest users’ funds. We present EVA²Hub, a PCH protocol that simultaneously supports variable-amount payments, two-sided value privacy, relationship anonymity, and robustness to griefing and aborts. EVA²Hub combines a double-blind signature scheme (DBSign) that authorizes arbitrary payment updates without letting the hub link payments to users, a double-base zero-knowledge range argument (DBZKRA) tailored to hidden channel balances, and a new PCH payment mechanism with resistance to aforementioned attacks. We formalize the security of EVA²Hub in the Universal Composability (UC) framework and prove that it realizes an ideal functionality capturing atomicity, value privacy, balance security, and anonymity against a malicious hub and corrupted users. A prototype implementation on Ethereum shows that EVA²Hub incurs low computational and communication overhead, making it practical for real-world deployment.

Index Terms—Payment channel hub, variable amount, value privacy, relationship anonymity, abort and griefing attacks.

I. SECURITY MODEL AND ANALYSIS OF DBSign

A. Security Model of DBSign

1) *Selective-Existential Unforgeability*: In Figure 1, we define the traditional unforgeability model and the selective-existential unforgeability model through the experiments $\text{Exp}_{\mathcal{A}, \text{DBSign}}^{\text{Forge}}(1^\lambda)$ and $\text{Exp}_{\mathcal{A}, \text{DBSign}}^{\text{S-Forge}}(1^\lambda)$, respectively. For the experiment $\text{Exp}_{\mathcal{A}, \text{DBSign}}^{\text{Forge}}(1^\lambda)$, $\mathcal{A}$ ’s advantage $\text{Adv}_{\text{DBSign}}^{\text{Forge}}$ is defined as $\Pr[\text{Exp}_{\mathcal{A}, \text{DBSign}}^{\text{Forge}}(1^\lambda) = 1]$. The scheme DBSign is said to be unforgeable if, for all PPT adversaries $\mathcal{A}$, the advantage $\text{Adv}_{\text{DBSign}}^{\text{Forge}}$ is negligible. Similarly, for the experiment $\text{Exp}_{\mathcal{A}, \text{DBSign}}^{\text{S-Forge}}(1^\lambda)$, the corresponding advantage $\text{Adv}_{\text{DBSign}}^{\text{S-Forge}}$ of the adversary $\mathcal{A}$ is defined as $\Pr[\text{Exp}_{\mathcal{A}, \text{DBSign}}^{\text{S-Forge}}(1^\lambda) = 1]$. The scheme DBSign is said to be selective-existentially unforgeable if, for all PPT adversaries $\mathcal{A}$, the advantage $\text{Adv}_{\text{DBSign}}^{\text{S-Forge}}$ is negligible. Moreover, the main difference between these two above experiments lies in the adversary $\mathcal{A}$ ’s capabilities. In the selective-existential unforgeability experiment, the adversary

$\mathcal{A}$ must declare the target message m^* in advance and is prohibited from sending it to the signing oracle $\mathcal{O}^S$ as a query. Notably, the oracle $\mathcal{O}^S$ differs from the Sign algorithm. The key distinction is that the adversary $\mathcal{A}$ submits (x, r_1, r_2) instead of the second-level blinded message m_{bb} as a query. However, this modification to $\mathcal{O}^S$ is reasonable. The main reason is that m_b and m_{bb} serve as the messages for the signer and the verifier, who invokes the algorithm Verify to check the validity of σ_b , respectively. Consequently, this modification does not compromise the selective-existential unforgeability of the scheme.

2) *First-Level Blindness*: We use the experiment in Figure 2 to formally define the property of first-level blindness. Specifically, the adversary $\mathcal{A}$ ’s advantage $\text{Adv}_{\text{DBSign}}^{\text{F-Blind}}$ is defined as $|\Pr[\text{Exp}_{\mathcal{A}, \text{DBSign}}^{\text{F-Blind}}(1^\lambda) = 1] - 1/2|$ within the context of the experiment $\text{Exp}_{\mathcal{A}, \text{DBSign}}^{\text{F-Blind}}(1^\lambda)$. The scheme DBSign is said to be first-level blinded if, for all PPT adversaries $\mathcal{A}$, the advantage $\text{Adv}_{\text{DBSign}}^{\text{F-Blind}}$ is negligible.

3) *Second-Level Blindness*: We use the experimental in Figure 3 to define the second-level blindness. Formally, the adversary $\mathcal{A}$ ’s advantage $\text{Adv}_{\text{DBSign}}^{\text{S-Blind}}$ is defined as $|\Pr[\text{Exp}_{\mathcal{A}, \text{DBSign}}^{\text{S-Blind}}(1^\lambda) = 1] - 1/2|$ in the experiment $\text{Exp}_{\mathcal{A}, \text{DBSign}}^{\text{S-Blind}}(1^\lambda)$. The scheme DBSign is said to be second-level blinded if, for all PPT adversaries $\mathcal{A}$, the advantage $\text{Adv}_{\text{DBSign}}^{\text{S-Blind}}$ is negligible.

B. Security Analysis of DBSign

Theorem 1. *The DBSign is selectively existentially unforgeable provided the Co-BDH+ assumption holds in the standard model.*

Proof. We will use the following game between the adversary $\mathcal{A}$ and the challenger $\mathcal{C}$ to prove this theorem, where we show that the adversary $\mathcal{A}$ forges a new signature without knowing sk is equivalent to solving the Co-BDH+ problem.

- **Init:** In this phase, the adversary $\mathcal{A}$ publishes the target message $m^* = x^*$ and the blind factor r_1^* which is used for generating the corresponding first-level blinded message $m_b^* = [x^*]G_1 + [r_1^*]Y_1$.
- **Setup:** In this phase, the challenger $\mathcal{C}$ first invokes the algorithm BSetup(1^λ) to acquire $(\mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T, \hat{\mathbb{G}}, q)$. Then, $\mathcal{C}$ samples generators $G \in \mathbb{G}$ and $\hat{G} \in \hat{\mathbb{G}}$, and sets $G_1 = [a]G$, $G_2 = [b]G$, $Y_1 = [-x^*/r_1^*]G_1 + [a]G$, and $Y_2 = [\beta]G$, where $\alpha, \beta \leftarrow_{\$} \mathbb{Z}_q^*$. In this case, $\mathcal{C}$ implicitly sets the private key $\text{sk} = a \cdot \gamma$ and the public key $\text{pk} = [\gamma]\hat{G}$, where $\gamma \leftarrow_{\$} \mathbb{Z}_q^*$. Finally, $\mathcal{C}$ publishes the public parameters $pp = (\mathbb{G}, \mathbb{G}_T, q, \hat{\mathbb{G}}, G_1, G_2, Y_1, Y_2, \hat{G})$ and the public key pk .

Corresponding Author: Jun Shao

X. Jia, P. Jiang, S. Zheng, and J. Shao are with the School of Computer Science and Technology, Zhejiang Gongshang University, Hangzhou, China 310018; and also with the Zhejiang Key Laboratory of Big Data and Future E-Commerce Technology, Hangzhou, China 310018. E-mail: chn.junshao@gmail.com.

C. Zuo is with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing, China.

This research is supported by National Natural Science Foundation of China (Grant No. 62272413, 62372040) and the Key Research and Development Program of Zhejiang under Grant 2025C02034.

$\text{Exp}_{\mathcal{A}, \text{DBSign}}^{\text{Forge}}(1^\lambda)$
1: $\mathcal{L} = \emptyset$ 2: $pp \leftarrow \text{Setup}(1^\lambda)$ 3: $(pk, sk) \leftarrow \text{KeyGen}(pp)$ 4: $(m_b^*, \sigma_b^*) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{S}_0}(\cdot)}(pk)$ 5: $a = \text{Verify}(m_b^*, \sigma_b^*, pk)$ 6: $c = m_b^* \notin \mathcal{L}$ 7: return $a \wedge c$
Oracle $\mathcal{O}^{\text{S}_0}(x)$
1: $(m_b, \sigma_b) \leftarrow \text{Sign}(\mathcal{C}(sk), \mathcal{A}(pk, x))$ 2: $\mathcal{L} \xleftarrow{\text{Add}} m_b$ 3: return (m_b, σ_b)
$\text{Exp}_{\mathcal{A}, \text{DBSign}}^{\text{S-Forge}}(1^\lambda)$
1: $(x^*, r_1^*, r_2^*) \leftarrow \mathcal{A}$ 2: $m_b^* \xleftarrow{\text{Blind}} (x^*, r_1^*)$ 3: $pp \leftarrow \text{Setup}(1^\lambda)$ 4: $(pk, sk) \leftarrow \text{KeyGen}(pp)$ 5: $\sigma_b^* \leftarrow \mathcal{A}^{\mathcal{O}^{\text{S}_1}(\cdot)}(pk)$ 6: return $\text{Verify}(m_b^*, \sigma_b^*, pk)$
Oracle $\mathcal{O}^{\text{S}_1}(x)$
1: $(m_b, \sigma_b) \leftarrow \text{Sign}(\mathcal{C}(sk), \mathcal{A}(pk, x))$ 2: if $m_b = m_b^*$ then return $\perp$ 3: else return (m_b, σ_b)

Fig. 1: Experiments for DBSign's unforgeability and selective-existential unforgeability.

- **Queries:** In this phase, the challenger $\mathcal{C}$ simulates the signing oracle $\mathcal{O}^{\text{S}}$ and initializes the list $\mathcal{L}$. Moreover, $\mathcal{A}$ can adaptively issue queries to the oracle $\mathcal{O}^{\text{S}}$.

Oracle $\mathcal{O}^{\text{S}}(x, r_1, r_2)$: Upon receiving a tuple (x, r_1, r_2) from $\mathcal{A}$ such that $m_b^* \neq [x]G_1 + [r_1]Y_1$, the challenger $\mathcal{C}$ proceeds as follows. It first selects a random element $w \in \mathbb{Z}_q^*$ and computes $\sigma_b^{(0)} = \left[\frac{-\alpha r_1 \gamma r_1^*}{x r_1^* - r_1 x^*} \right] G_2 + [w] \left(\left[x - \frac{r_1}{r_1^*} x^* \right] G_1 + [\alpha \cdot r_1] G \right)$. To simplify the expression, define $w' = w - \frac{b \gamma r_1^*}{x r_1^* - r_1 x^*}$. Substituting w' into the equation yields $\sigma_b^{(0)} = [a \cdot \gamma] G_2 + [w'] ([x]G_1 + [r_1]Y_1)$. Next, $\mathcal{C}$ computes $T = [w']G = [w]G + \left[\frac{-\gamma r_1^*}{x r_1^* - r_1 x^*} \right] G_2$, $\sigma_{bb}^{(2)} = [w']Y_2 = [w' \cdot \beta]G = \beta \cdot T$. Then, $\sigma_{bb}^{(0)} = [a \cdot \gamma]G_2 + [w']([x]G_1 + [r_1]Y_1 + [r_2]Y_2) = \sigma_b^{(0)} + r_2 \cdot \sigma_{bb}^{(2)}$. After that, $\sigma_{bb}^{(1)} = [w']\hat{G} = [w]\hat{G} + \left[\frac{-\gamma r_1^*}{x r_1^* - r_1 x^*} \right] [b]\hat{G}$. Finally, the challenger $\mathcal{C}$ outputs the second-level blinded signature $\sigma_{bb} = (\sigma_{bb}^{(0)}, \sigma_{bb}^{(1)}, \sigma_{bb}^{(2)})$.

- **Output:** The adversary $\mathcal{A}$ outputs a first-level bind signature σ_b^* . We say that $\mathcal{A}$ wins the game if σ_b^* is a valid signature on the designated m_b^* and the public key pk .

$\text{Exp}_{\mathcal{A}, \text{DBSign}}^{\text{F-Blind}}(1^\lambda)$
1: $\mathcal{L} = \emptyset$ 2: $pp \leftarrow \text{Setup}(1^\lambda)$ 3: $(x_0, x_1, pk) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{M-Rand}}(\cdot), \mathcal{O}^{\text{S-Ubld}}(\cdot)}(pp)$ 4: $a \leftarrow_{\$} \{0, 1\}$ 5: $(m_{ba}, \sigma_{ba}) \leftarrow \text{Sign}(\mathcal{A}(sk), \mathcal{C}(pk, x_a))$ 6: $a' \leftarrow \mathcal{A}^{\mathcal{O}^{\text{M-Rand}}(\cdot), \mathcal{O}^{\text{S-Ubld}}(\cdot)}(m_{ba}, \sigma_{ba})$ 7: return $a \stackrel{?}{=} a'$
Oracle $\mathcal{O}^{\text{M-Rand}}(x)$
1: $m_{bb} \xleftarrow{\text{Search } (x, *, *, *, m_{bb})} \mathcal{L}$ 2: return m_{bb} if m_{bb} exists 3: $r_1, r_2 \leftarrow_{\$} \mathcal{R}$ // $\mathcal{R}$ is the randomness space 4: $(m_b, m_{bb}) \xleftarrow{\text{Blind}} (x, r_1, r_2)$ 5: $\mathcal{L} \xleftarrow{\text{Add}} (x, r_1, r_2, m_b, m_{bb})$ 6: return m_{bb}
Oracle $\mathcal{O}^{\text{S-Ubld}}(m_{bb}, \sigma_{bb})$
1: $(r_2, m_b) \xleftarrow{\text{Search } (*, *, r_2, m_b, m_{bb})} \mathcal{L}$ 2: return $\perp$ if r_2 not exists 3: $\tilde{w} \leftarrow_{\$} \mathcal{R}$ 4: $\sigma_b \xleftarrow{\text{Unblind}} (\sigma_{bb}, m_b, r_2, \tilde{w})$ 5: return (m_b, σ_b)

Fig. 2: Experiment for DBSign's first-level blindness.

If the adversary $\mathcal{A}$ wins the game, it outputs a valid signature

$$\begin{aligned} \sigma_b^* &= (\sigma_b^{(0)} = [a \cdot \gamma]G_2 + [w' + \tilde{w}]([x^*]G_1 + [r_1^*]Y_1), \\ &\quad \sigma_b^{(1)} = [w' + \tilde{w}]\hat{G}, \\ &\quad \sigma_b^{(2)} = [w' + \tilde{w}]Y_2) \end{aligned}$$

on m_b^* and pk , which means the equations $\hat{e}(\sigma_b^{(0)}, \hat{G}) = \hat{e}(G_2, pk) \cdot \hat{e}(m_b, \sigma_b^{(1)})$ hold. In this case, we could obtain the Co-BDH+ solution:

$$\begin{aligned} [a \cdot b]G &= \frac{1}{\gamma} \left(\sigma_b^{(0)} - \frac{\alpha r_1^*}{\beta} \sigma_b^{(1)} \right) \\ &= \frac{1}{\gamma} ([a \cdot \gamma]G_2 + [w' + \tilde{w}]([x^*]G_1 + [r_1^*]Y_1) \\ &\quad - [\alpha \cdot r_1^*][w' + \tilde{w}][1/\beta]Y_2) \\ &= \frac{1}{\gamma} ([a \cdot \gamma]G_2 + [w' + \tilde{w}]([x^*]G_1 + [r_1^*]Y_1) \\ &\quad - [\alpha \cdot r_1^*][w' + \tilde{w}][1/\beta][\beta]G) \\ &= \frac{1}{\gamma} [a \cdot \gamma]G_2 \end{aligned}$$

Hence, we obtain this theorem. $\square$

Theorem 2. The DBSign is first-level blinded for all adversaries $\mathcal{A}$ in the standard model.

Proof. We prove this theorem through the following game between an adversary $\mathcal{A}$ and a challenger $\mathcal{C}$, where we show that the advantage $\text{Adv}_{\text{DBSign}}^{\text{F-Blind}}$ of any adversary $\mathcal{A}$, even with unlimited computational resources, is negligible.

$\text{Exp}_{\mathcal{A}, \text{DBSign}}^{\text{S-Blind}}(1^\lambda)$
<pre> 1 : $\mathcal{L} = \emptyset$ 2 : $pp \leftarrow \text{Setup}(1^\lambda)$ 3 : $(x_0, r_1, x_1, r_2, pk) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{M-Rand}}(\cdot), \mathcal{O}^{\text{S-Ubld}}(\cdot)}(pp)$ 4 : $a \leftarrow_{\\$} \{0, 1\}$ 5 : $\sigma_{bba} \leftarrow \text{Sign}(\mathcal{A}(sk), \mathcal{C}(pk, x_a))$ 6 : $m_{ba} \xleftarrow{\text{Blind}} (x_a, r_{1a})$ 7 : $m_{bba} \xleftarrow{\text{Blind}} (m_{ba}, r_{2a})$ // r_{2a} is the blind factor used in step 5 8 : $a' \leftarrow \mathcal{A}^{\mathcal{O}^{\text{M-Rand}}(\cdot), \mathcal{O}^{\text{S-Ubld}}(\cdot)}(m_{bba}, \sigma_{bba})$ 9 : return $a \stackrel{?}{=} a'$ </pre>
Oracle $\mathcal{O}^{\text{M-Rand}}(x)$ <pre> 1 : $m_{bb} \xleftarrow{\text{Search}(x, *, *, *, m_{bb})} \mathcal{L}$ 2 : return m_{bb} if m_{bb} exists 3 : $r_1, r_2 \leftarrow_{\\$} \mathcal{R}$ // $\mathcal{R}$ is the randomness space 4 : $(m_b, m_{bb}) \xleftarrow{\text{Blind}} (x, r_1, r_2)$ 5 : $\mathcal{L} \xleftarrow{\text{Add}} (x, r_1, r_2, m_b, m_{bb})$ 6 : return m_{bb} </pre>
Oracle $\mathcal{O}^{\text{S-Ubld}}(m_{bb}, \sigma_{bb})$ <pre> 1 : $(r_2, m_b) \xleftarrow{\text{Search}(*, *, r_2, m_b, m_{bb})} \mathcal{L}$ 2 : return $\perp$ if r_2 not exists 3 : $\tilde{w} \leftarrow_{\\$} \mathcal{R}$ 4 : $\sigma_b \xleftarrow{\text{Unblind}} (\sigma_{bb}, m_b, r_2, \tilde{w})$ 5 : return (m_b, σ_b) </pre>

Fig. 3: Experiment for DBSign's second-level blindness.

- **Setup:** The challenger $\mathcal{C}$ first invokes the algorithm $\text{BSetup}(1^\lambda)$ to generate $(\mathbb{G}, \mathbb{G}_T, q, \hat{e})$. After that, it selects the random generators G, G_1, G_2, Y_1, Y_2 from the group $\mathbb{G}$ and publishes the public parameter $pp = (\mathbb{G}, \mathbb{G}_T, q, \hat{e}, G_1, G_2, Y_1, Y_2, \hat{G})$. Finally, the adversary $\mathcal{A}$ selects $sk \leftarrow_{\$} \mathbb{Z}_q^*$, computes the public key $pk = [sk]\hat{G}$, and publishes pk .
- **Phase 1:** In this phase, the challenger $\mathcal{C}$ simulates the oracles $\mathcal{O}^{\text{M-Rand}}$ and $\mathcal{O}^{\text{S-Ubld}}$, and initializes the list $\mathcal{L}$. Moreover, the adversary $\mathcal{A}$ can adaptively issue queries to these oracles.

Oracle $\mathcal{O}^{\text{M-Rand}}(x)$: Inputting x from $\mathcal{A}$, $\mathcal{C}$ first searches for an entry $(x, *, *, *, m_{bb})$ in the list $\mathcal{L}$. If such an entry exists, $\mathcal{C}$ outputs m_{bb} as the response. Otherwise, $\mathcal{C}$ selects $r_1, r_2 \leftarrow_{\$} \mathbb{Z}_q^*$ and computes the first-level blinded message $m_b = [x]G_1 + [r_1]Y_1$ and the second-level blinded message $m_{bb} = m_b + [r_2]Y_2$. Finally, $\mathcal{C}$ outputs m_{bb} and records $(x, r_1, r_2, m_b, m_{bb})$ in the list $\mathcal{L}$.

Oracle $\mathcal{O}^{\text{S-Ubld}}(m_{bb}, \sigma_{bb})$: Inputting the second-level blinded message m_{bb} and the corresponding signature σ_{bb} from $\mathcal{A}$, $\mathcal{C}$ first searches for an entry $(*, *, r_2, m_b, m_{bb})$ in the list $\mathcal{L}$. If such an entry not exists, $\mathcal{C}$ aborts. Otherwise, $\mathcal{C}$ checks whether the following equations hold: $\hat{e}(\sigma_{bb}^{(0)}, \hat{G}) \stackrel{?}{=} \hat{e}(G_2, pk) \cdot \hat{e}(m_{bb}, \sigma_{bb}^{(1)})$ and $\hat{e}(Y_2, \sigma_{bb}^{(1)}) \stackrel{?}{=} \hat{e}(\sigma_{bb}^{(2)}, \hat{G})$. If either of these equations fails, $\mathcal{C}$ terminates

the process. Otherwise, $\mathcal{C}$ randomly selects $\tilde{w} \in \mathbb{Z}_q^*$ and computes the first-level blinded signature $\sigma_b = (\sigma_b^{(0)}, \sigma_b^{(1)}, \sigma_b^{(2)})$, where $\sigma_b^{(0)} = \sigma_{bb}^{(0)} - r_2 \cdot \sigma_{bb}^{(2)} + \tilde{w} \cdot m_b = [sk]G_2 + (w + \tilde{w})m_b$, $\sigma_b^{(1)} = \sigma_{bb}^{(1)} + [\tilde{w}]\hat{G} = [w + \tilde{w}]\hat{G}$, and $\sigma_b^{(2)} = \sigma_{bb}^{(2)} + [\tilde{w}]Y_2 = [w + \tilde{w}]Y_2$. Finally, $\mathcal{C}$ outputs (m_b, σ_b) .

- **Challenge:** In this phase, the adversary $\mathcal{A}$ randomly selects x_0 and x_1 , and sends (x_0, x_1) to $\mathcal{C}$. After that, $\mathcal{C}$ generates $a \leftarrow_{\$} \{0, 1\}$ and selects elements $r_{1a}, r_{2a} \leftarrow_{\$} \mathbb{Z}_q^*$. $\mathcal{C}$ then computes the first-level blinded message $m_{ba} = [x_a]G_1 + [r_{1a}]Y_1$ and the second-level blinded message $m_{bba} = m_{ba} + [r_{2a}]Y_2$. $\mathcal{C}$ then sends m_{bba} to $\mathcal{A}$ for the corresponding second-level blinded signature σ_{bba} . Specifically, $\mathcal{A}$ selects an element $w_a \leftarrow_{\$} \mathbb{Z}_q^*$, and computes the signature $\sigma_{bba} = (\sigma_{bba}^{(0)}, \sigma_{bba}^{(1)}, \sigma_{bba}^{(2)})$ with the secret key sk , where $\sigma_{bba}^{(0)} = [sk]G_2 + w_a \cdot m_{bba}$, $\sigma_{bba}^{(1)} = [w_a]\hat{G}$, and $\sigma_{bba}^{(2)} = [w_a]Y_2$. $\mathcal{A}$ then sends σ_{bba} to $\mathcal{C}$. Subsequently, $\mathcal{C}$ selects $\tilde{w}_a \leftarrow_{\$} \mathbb{Z}_q^*$ and computes the first-level blinded signature $\sigma_{ba} = (\sigma_{ba}^{(0)}, \sigma_{ba}^{(1)}, \sigma_{ba}^{(2)})$, where $\sigma_{ba}^{(0)} = \sigma_{bba}^{(0)} - r_{2a} \cdot \sigma_{bba}^{(2)} + \tilde{w}_a \cdot m_{ba}$, $\sigma_{ba}^{(1)} = \sigma_{bba}^{(1)} + [\tilde{w}_a]\hat{G} = [w_a + \tilde{w}_a]\hat{G}$, and $\sigma_{ba}^{(2)} = \sigma_{bba}^{(2)} + [\tilde{w}_a]Y_2 = [w_a + \tilde{w}_a]Y_2$.
- **Phase 2:** The adversary $\mathcal{A}$ proceeds as in Phase 1.
- **Guess:** The adversary outputs a guess a' on a . If $a' = a$, we say that the adversary wins the game.

For all adversaries $\mathcal{A}$, even with unlimited computational resources, their advantage $\text{Adv}_{\text{DBSign}}^{\text{F-Blind}}$ in winning this game is 0. This is because, for the first-level blinded message m_{ba} where $a \in \{0, 1\}$, there always exist $r_{1a}, r_{1(1-a)} \in \mathbb{Z}_q^*$ that satisfy the conditions $m_b = [x_0]G_1 + [r_{1a}]Y_1 = [x_1]G_1 + [r_{1(1-a)}]Y_1$. As a result, the probability of any adversary $\mathcal{A}$ winning this game is 1/2.

Hence, we obtain this theorem. $\square$

Theorem 3. *The DBSign is second-level blinded for all adversaries $\mathcal{A}$ in the standard model.*

Proof. We prove this theorem through the following game between an adversary $\mathcal{A}$ and a challenger $\mathcal{C}$, where we show that the advantage $\text{Adv}_{\text{DBSign}}^{\text{S-Blind}}$ of any adversary $\mathcal{A}$, even with unlimited computational resources, is negligible.

- **Setup:** Consistent with the first-level blindness model.
- **Phase 1:** Consistent with the first-level blindness model.
- **Challenge:** In this phase, the adversary $\mathcal{A}$ randomly selects $(x_0, r_{10}, x_1, r_{11})$, and sends them to $\mathcal{C}$. After that, $\mathcal{C}$ selects a random bit $a \in \{0, 1\}$ and computes the first-level blinded message $m_{ba} = [x_a]G_1 + [r_{1a}]Y_1$. $\mathcal{C}$ then chooses $r_{2a} \leftarrow_{\$} \mathbb{Z}_q^*$, computes the second-level blinded message $m_{bba} = m_{ba} + [r_{2a}]Y_2$. Finally, $\mathcal{C}$ outputs m_{bba} .
- **Phase 2:** Consistent with the first-level blindness model.
- **Guess:** The adversary outputs a guess a' on a . If $a' = a$, we say that the adversary wins the game.

For all adversaries $\mathcal{A}$, even with unlimited computational resources, their advantage $\text{Adv}_{\text{DBSign}}^{\text{S-Blind}}$ in winning this game is 0. This is because, for the second-level blinded message m_{bba} where $a \in \{0, 1\}$, there always exist $r_{2a}, r_{2(1-a)} \in \mathbb{Z}_q^*$ that satisfy the conditions $m_{bba} = m_{ba} + [r_{2a}]Y_2 = m_{ba} + [r_{2(1-a)}]Y_2$.

$\text{Exp}_{\mathcal{A}, \text{DBRSig}^{\text{Forge}}}^{\text{Forge}}(1^\lambda)$
1: $\mathcal{L}_0 = \emptyset, \mathcal{L}_1 = \emptyset$ 2: $pp \leftarrow \text{Setup}(1^\lambda)$ 3: $\{pk_i, sk_i\}_{i=1}^{n(1^\lambda)} \leftarrow \text{KeyGen}(pp)$ // $n(1^\lambda)$ is a polynomial with 1^λ as input 4: $\mathcal{PK} = \{pk_i\}_{i=1}^{n(1^\lambda)}$ 5: $(m^*, \sigma^*, \mathcal{Y}^*) \leftarrow \mathcal{A}^{\mathcal{O}^C(\cdot), \mathcal{O}^S(\cdot)}(\mathcal{PK})$ 6: $a = \text{Verify}(\sigma^*, m^*, \mathcal{Y}^*)$ 7: return $a \wedge (m^* \notin \mathcal{L}_1) \wedge (\mathcal{Y}^* \cap \mathcal{L}_0 = \emptyset)$ Oracle $\mathcal{O}^C(pk)$ 1: return $\perp$ if $pk \notin \mathcal{PK}$ 2: $\mathcal{L}_0 \xleftarrow{\text{Add}} pk$ 3: return sk // sk 's corresponding public key is pk Oracle $\mathcal{O}^S(m, \mathcal{Y})$ 1: return $\perp$ if $\mathcal{Y} \not\subseteq \mathcal{PK}$ 2: $\pi \leftarrow_{\$} \{1, \dots, \mathcal{Y} \}$ // π should satisfy $pk_\pi \notin \mathcal{L}_0$ 3: $\sigma \leftarrow \text{Sign}(m, \mathcal{Y}, sk_\pi)$ 4: $\mathcal{L}_1 \xleftarrow{\text{Add}} m$ 5: return σ

Fig. 4: Experiment for DBRSig's unforgeability.

$[r_{2(1-a)}]Y_2$. As a result, the probability of any adversary $\mathcal{A}$ winning this game is $1/2$.

Hence, we obtain this theorem. $\square$

II. SECURITY MODEL AND ANALYSIS OF DBRSig

A. Security Model of DBRSig

1) *Security Model.*: Security of scheme DBRSig has two aspects: *unforgeability* and *anonymity*. Each of these aspects is formally defined using the experiments shown in Figure 4 and Figure 5, respectively.

Unforgeability. The unforgeability model of the DBRSig scheme is formally defined using the experiment in Figure 4. Specifically, $\mathcal{A}$'s advantage $\text{Adv}_{\text{DBRSig}^{\text{Forge}}}^{\text{Forge}}$ is defined as $\Pr[\text{Exp}_{\mathcal{A}, \text{DBRSig}^{\text{Forge}}}^{\text{Forge}}(1^\lambda) = 1]$. The DBRSig scheme is considered unforgeable if, for all PPT adversaries $\mathcal{A}$, the advantage $\text{Adv}_{\text{DBRSig}^{\text{Forge}}}^{\text{Forge}}$ is negligible.

Anonymity. We use the experiment in Figure 5 to formally define the property of anonymity. Specifically, the adversary $\mathcal{A}$'s advantage $\text{Adv}_{\text{DBRSig}^{\text{Anon}}}^{\text{Anon}}$ is defined as $|\Pr[\text{Exp}_{\mathcal{A}, \text{DBRSig}^{\text{Anon}}}^{\text{Anon}}(1^\lambda) = 1] - 1/2|$ within the context of the experiment $\text{Exp}_{\mathcal{A}, \text{DBRSig}^{\text{Anon}}}^{\text{Anon}}(1^\lambda)$. The scheme DBRSig is said to be anonymity if, for all PPT adversaries $\mathcal{A}$, the advantage $\text{Adv}_{\text{DBRSig}^{\text{Anon}}}^{\text{Anon}}$ is negligible.

B. Security Analysis of DBRSig

Theorem 4. *The DBRSig is unforgeable for all PPT adversaries $\mathcal{A}$ provided the DL assumption holds in the random oracle model.*

Proof. We will use the game between the adversary $\mathcal{A}$ and the challenger $\mathcal{C}$ to prove this theorem. Specifically, we show that

$\text{Exp}_{\mathcal{A}, \text{DBRSig}^{\text{Anon}}}^{\text{Anon}}(1^\lambda)$
1: $\mathcal{L} = \emptyset$ 2: $pp \leftarrow \text{Setup}(1^\lambda)$ 3: $\{pk_i, sk_i\}_{i=1}^{n(1^\lambda)} \leftarrow \text{KeyGen}(pp)$ // $n(1^\lambda)$ is a polynomial with 1^λ as input 4: $\mathcal{PK} = \{pk_i\}_{i=1}^{n(1^\lambda)}$ 5: $(pk_0^*, pk_1^*, m^*) \leftarrow \mathcal{A}^{\mathcal{O}^C(\cdot), \mathcal{O}^S(\cdot)}(\mathcal{PK})$ // $pk_0^*, pk_1^* \in \mathcal{PK}$ 6: $\mathcal{Y}^* = (pk_0^*, pk_1^*)$ 7: $a \leftarrow_{\$} \{0, 1\}$ 8: $\sigma_a^* \leftarrow \text{Sign}(m^*, sk_a, \mathcal{Y}^*)$ 9: $a' \leftarrow \mathcal{A}^{\mathcal{O}^C(\cdot), \mathcal{O}^S(\cdot)}(\mathcal{PK})$ 10: return $a \stackrel{?}{=} a'$ Oracle $\mathcal{O}^C(pk)$ 1: return $\perp$ if $pk \notin \mathcal{PK}$ 2: $\mathcal{L} \xleftarrow{\text{Add}} pk$ 3: return sk // sk 's corresponding public key is pk Oracle $\mathcal{O}^S(m, \mathcal{Y})$ 1: return $\perp$ if $\mathcal{Y} \not\subseteq \mathcal{PK}$ 2: $\pi \leftarrow_{\$} \{1, \dots, \mathcal{Y} \}$ // π should satisfy $pk_\pi \notin \mathcal{L}$ 3: $\sigma \leftarrow \text{Sign}(m, \mathcal{Y}, sk_\pi)$ 4: return σ

Fig. 5: Experiment for DBRSig's anonymity.

the adversary $\mathcal{A}$ forges a new signature without knowing the secret key sk is equivalent to solve DL problem by introducing the Rewind-on-Success (RoS) lemma. We omit the illustration for the RoS and refer readers to [1] for further details.

- **Setup:** The challenger $\mathcal{C}$ first generates the public parameter $pp = (\mathbb{G}, q, Y_1, Y_2, H_1, H_2)$, where $\mathbb{G}$ is a group of prime order q , Y_1 is the random generator in $\mathbb{G}$, and $Y_2 = [\alpha]Y_1$ ($\alpha \leftarrow_{\$} \mathbb{Z}_q^*$). $\mathcal{C}$ then sets $Z = [\alpha]Y_1$, where $[\alpha]Y_1$ is the DL problem. Subsequently, $\mathcal{C}$ generates the public key list $\mathcal{PK} = \{pk_i = [w_i](Z + [\alpha \cdot \gamma]Z)\}_{i=1}^{n(1^\lambda)}$, where w_i is the random element in $\mathbb{Z}_q^*$, and $n(1^\lambda)$ is a polynomial with 1^λ as input. Finally, $\mathcal{C}$ publishes pp and $\mathcal{PK}$.

- **Queries:** In this phase, the challenger $\mathcal{C}$ simulates the following oracles $\mathcal{O}^{H_1}$, $\mathcal{O}^{H_2}$ and $\mathcal{O}^S$ and the adversary $\mathcal{A}$ can adaptively issue queries to these oracles. Moreover, $\mathcal{C}$ initializes the lists $\mathcal{L}_0$, $\mathcal{L}_1$ and $\mathcal{L}_2$ as empty lists.

Oracle $\mathcal{O}^{H_1}(\mathcal{Y})$: Upon receiving a public key list $\mathcal{Y}$ from $\mathcal{A}$, $\mathcal{C}$ first searches for an entry $(\mathcal{Y}, \star, H_1(\mathcal{Y}, 1), H_1(\mathcal{Y}, 2))$ in the list $\mathcal{L}_0$. If such an entry exists, $\mathcal{C}$ outputs $(Y_1 = H_1(\mathcal{Y}, 1), \tilde{Y}_2 = H_1(\mathcal{Y}, 2))$ as the response. Otherwise, $\mathcal{C}$ generates $r \leftarrow_{\$} \mathbb{Z}_q^*$ and sets $H_1(\mathcal{Y}, 1) = [r]Y_1$ and $H_1(\mathcal{Y}, 2) = [r]Y_2$. Finally, $\mathcal{C}$ outputs $(\tilde{Y}_1 = H_1(\mathcal{Y}, 1), \tilde{Y}_2 = H_1(\mathcal{Y}, 2))$ and records $(\mathcal{Y}, r, H_1(\mathcal{Y}, 1), H_1(\mathcal{Y}, 2))$ in the list $\mathcal{L}_0$.

Oracle $\mathcal{O}^{H_2}(\mathcal{Y}, m, z_1, z_2)$: Upon receiving a public key list $\mathcal{Y}$, a message m , and two elements $(z_1, z_2) \in \mathbb{G}^2$ from $\mathcal{A}$, $\mathcal{C}$ first searches for an entry

$(\mathcal{Y}, m, z_1, z_2, H_2(\mathcal{Y}, m, z_1, z_2))$ in the list $\mathcal{L}_1$. If such an entry exists, $\mathcal{C}$ outputs $H_2(\mathcal{Y}, m, z_1, z_2)$ as the response. Otherwise, $\mathcal{C}$ randomly selects v from $\mathbb{Z}_q^*$ and sets $H_2(\mathcal{Y}, m, z_1, z_2) = v$. Finally, $\mathcal{C}$ outputs $H_2(\mathcal{Y}, m, z_1, z_2)$ and records $(\mathcal{Y}, m, z_1, z_2, H_2(\mathcal{Y}, m, z_1, z_2))$ in the list $\mathcal{L}_1$.

Oracle $\mathcal{O}^S(m, \mathcal{Y})$: Upon receiving a message m and a public key list $\mathcal{Y} \subseteq \mathcal{PK}$ from $\mathcal{A}$, $\mathcal{C}$ first acquires $(\tilde{Y}_1 = [r]Y_1, \tilde{Y}_2 = [r]Y_2)$ by querying $\mathcal{O}^{H_1}$ with $\mathcal{Y}$ and randomly selects $\pi \in \{1, \dots, \ell\}$, where ℓ is the size of $\mathcal{Y}$. Then, $\mathcal{C}$ generates $r \leftarrow_{\$} \mathbb{Z}_q^*$ and computes $\tilde{pk} = r \cdot pk_\pi$, implicitly indicating that $\mathcal{C}$ attempts to output a signature σ signed with the secret key sk_π . For $i \in \{1, \dots, \ell\}$, $\mathcal{C}$ computes $z_{i1} = [\beta_i]Y_1 + v_i \tilde{pk} + [\beta'_i]Y_2$ and $z_{i2} = [\beta_i]\tilde{Y}_1 + v_i \tilde{pk} + [\beta'_i]\tilde{Y}_2$, where $c_i, v_i, v_{i+1}, \beta_i, \beta'_i \leftarrow_{\$} \mathbb{Z}_q^*$. Subsequently, $\mathcal{C}$ sets $H_2(\mathcal{Y}, m, z_{i1}, z_{i2}) = v_{i+1}$ and records $(\mathcal{Y}, m, z_i, H_2(\mathcal{Y}, m, z_{i1}, z_{i2}))$ in the list $\mathcal{L}_1$, and m in the list $\mathcal{L}_2$. Finally, $\mathcal{C}$ outputs the signature $\sigma = (v_1, \beta_1, \dots, \beta_\ell, \beta'_1, \dots, \beta'_\ell, \tilde{pk})$ to complete this query.

- **Output:** The adversary outputs $(\sigma^*, m^*, \mathcal{Y}^*)$. The adversary $\mathcal{A}$ is considered to win the game if both the following conditions are satisfied: (1) $m^* \notin \mathcal{L}_2$, (2) $1 = \text{Verify}(\sigma^*, m^*, \mathcal{Y}^*)$.

Similar to [1], we assume that the adversary $\mathcal{A}$ makes at most q_H queries to the oracles $\mathcal{O}^{H_1}$ and $\mathcal{O}^{H_2}$ combined, and at most q_S queries to the oracle $\mathcal{O}^S$. Meanwhile, q_H and q_S grow at most polynomially with respect to 1^λ . We leverage the conclusion in [1]: if the adversary $\mathcal{A}$ can forge a valid signature to win the above game with a probability no less than $1/(\ell(q_H + \ell q_S)Q(k))$, we can rewind the adversary $\mathcal{A}$ to forge two different but valid elements $(\sigma^*, m^*, \mathcal{Y}^*)$ and $(\bar{\sigma}^*, m^*, \mathcal{Y}^*)$ with at least a non-negligible probability $1/(\ell(q_H + \ell q_S)Q(k))^2$. Here, $Q(k)$ is a sufficiently large polynomial, $\sigma^* = (v_1^*, \beta_1^*, \dots, \beta_\ell^*, \beta'_1^*, \dots, \beta'_\ell^*, \tilde{pk})$, and $\bar{\sigma}^* = (\bar{v}_1^*, \bar{\beta}_1^*, \dots, \bar{\beta}_\ell^*, \bar{\beta}'_1^*, \dots, \bar{\beta}'_\ell^*, \tilde{pk})$, satisfying the equation: $[\beta_\pi^*]Y_1 + v_{\pi-1}^* pk_\pi + [\beta'_\pi^*]Y_2 = [\bar{\beta}_\pi^*]Y_1 + \bar{v}_{\pi-1}^* pk_\pi + [\bar{\beta}'_\pi^*]Y_2$. For simplicity, we set $E_1 = [\beta_\pi^*]Y_1 + v_{\pi-1}^* pk_\pi + [\beta'_\pi^*]Y_2$ and $E_2 = [\bar{\beta}_\pi^*]Y_1 + \bar{v}_{\pi-1}^* pk_\pi + [\bar{\beta}'_\pi^*]Y_2$. It is worth noting that σ^* and $\bar{\sigma}^*$ are associated with the same secret key sk_π . In this case, we can use the outputs of the adversary $\mathcal{A}$ to derive a solution a to the DL problem based on the following equations:

$$\begin{aligned} E_1 &= [\beta_\pi^*]Y_1 + [v_{\pi-1}^* \cdot w_\pi]([a]Y_1 + [a]Y_2) + [\beta'_\pi^*]Y_2 \\ &= [\beta_\pi^*]Y_1 + [v_{\pi-1}^* \cdot w_\pi]([a]Y_1 + [a \cdot \alpha \cdot \gamma]Y_1) + [\beta'_\pi^* \cdot \alpha]Y_1 \\ &= [\beta_\pi^* + v_{\pi-1}^* \cdot w_\pi^* \cdot a + v_{\pi-1}^* \cdot w_\pi \cdot a \cdot \alpha \cdot \gamma + \beta'_\pi^* \cdot \alpha]Y_1, \\ E_2 &= [\bar{\beta}_\pi^*]Y_1 + [\bar{v}_{\pi-1}^* \cdot w_\pi]([a]Y_1 + [a]Y_2) + [\bar{\beta}'_\pi^*]Y_2 \\ &= [\bar{\beta}_\pi^*]Y_1 + [\bar{v}_{\pi-1}^* \cdot w_\pi]([a]Y_1 + [a \cdot \alpha \cdot \gamma]Y_1) + [\bar{\beta}'_\pi^* \cdot \alpha]Y_1 \\ &= [\bar{\beta}_\pi^* + \bar{v}_{\pi-1}^* \cdot w_\pi^* \cdot a + \bar{v}_{\pi-1}^* \cdot w_\pi \cdot a \cdot \alpha \cdot \gamma + \bar{\beta}'_\pi^* \cdot \alpha]Y_1, \\ a &= \frac{\bar{\beta}_\pi^* + \alpha \bar{\beta}'_\pi^* - \beta_\pi^* - \alpha \beta'_\pi^*}{w_\pi v_{\pi-1}^* + \alpha \gamma w_\pi v_{\pi-1}^* - w_\pi \bar{v}_{\pi-1}^* - \alpha \gamma w_\pi \bar{v}_{\pi-1}^*}. \end{aligned}$$

Hence, we obtain this theorem. $\square$

Theorem 5. The DBRSig is anonymous for all PPT adversaries $\mathcal{A}$ provided the DDH assumption holds in the random oracle model.

Proof. We will use the game between the adversary $\mathcal{A}$ and the challenger $\mathcal{C}$ to prove this theorem. Specifically, we show that the ability of the adversary $\mathcal{A}$ to distinguish the signer of a group, given the message m , the signature σ , and the corresponding public key list $\mathcal{Y}$, is equivalent to solving the DDH problem.

- **Setup:** The challenger $\mathcal{C}$ first generates the public parameter $pp = (\mathbb{G}, q, Y_1, Y_1, H_1, H_2)$, where $\mathbb{G}$ is a group of prime order q , Y_1 is the random generator in the group $\mathbb{G}$, and $Y_2 = [\alpha]Y_1 (\alpha \leftarrow_{\$} \mathbb{Z}_q^*)$. $\mathcal{C}$ then generates the public key list $\mathcal{PK} = \{pk_i = [c_i] \cdot [a]Y_1 + [c'_i \cdot \alpha] \cdot [a]Y_1\}_{i=1}^{n(1^\lambda)}$, where $n(1^\lambda)$ is a polynomial with 1^λ as input, c_i and c'_i are the random elements in $\mathbb{Z}_p^*$. In this case, we implicitly set $pk_i = [a \cdot c_i]Y_1 + [a \cdot c'_i]Y_2$ and $sk_i = (sk_0 = ac_i, sk_1 = ac'_i)$. Finally, $\mathcal{C}$ publishes pp and $\mathcal{PK}$.
- **Phase 1:** In this phase, the challenger $\mathcal{C}$ simulates the following oracles $\mathcal{O}^{H_1}$, $\mathcal{O}^{H_2}$ and $\mathcal{O}^S$ and the adversary $\mathcal{A}$ can adaptively issue queries to these oracles. Moreover, $\mathcal{C}$ initializes the lists $\mathcal{L}_0$ and $\mathcal{L}_1$ as empty lists.

Oracle $\mathcal{O}^{H_1}(\mathcal{Y})$: Upon receiving a public key list $\mathcal{Y}$ from $\mathcal{A}$, $\mathcal{C}$ first searches for an entry $(\mathcal{Y}, *, H_1(\mathcal{Y}, 1), H_1(\mathcal{Y}, 2))$ in the list $\mathcal{L}_0$. If such an entry exists, $\mathcal{C}$ outputs $(\tilde{Y}_1 = H_1(\mathcal{Y}, 1), \tilde{Y}_2 = H_1(\mathcal{Y}, 2))$ as the response. Otherwise, $\mathcal{C}$ generates $r \leftarrow_{\$} \mathbb{Z}_q^*$ and sets $H_1(\mathcal{Y}, 1) = [r]Y_1$ and $H_1(\mathcal{Y}, 2) = [r]Y_2$. Finally, $\mathcal{C}$ outputs $(\tilde{Y}_1 = H_1(\mathcal{Y}, 1), \tilde{Y}_2 = H_1(\mathcal{Y}, 2))$ and records $(\mathcal{Y}, r, H_1(\mathcal{Y}, 1), H_1(\mathcal{Y}, 2))$ in the list $\mathcal{L}_0$.

Oracle $\mathcal{O}^{H_2}(\mathcal{Y}, m, z_1, z_2)$: Upon receiving a public key list $\mathcal{Y}$, a message m , and two elements $(z_1, z_2) \in \mathbb{G}^2$ from $\mathcal{A}$, $\mathcal{C}$ first searches for an entry $(\mathcal{Y}, m, z_1, z_2, H_2(\mathcal{Y}, m, z_1, z_2))$ in the list $\mathcal{L}_1$. If such an entry exists, $\mathcal{C}$ outputs $H_2(\mathcal{Y}, m, z_1, z_2)$ as the response. Otherwise, $\mathcal{C}$ randomly selects v from $\mathbb{Z}_q^*$ and sets $H_2(\mathcal{Y}, m, z_1, z_2) = v$. Finally, $\mathcal{C}$ outputs $H_2(\mathcal{Y}, m, z_1, z_2)$ and records $(\mathcal{Y}, m, z_1, z_2, H_2(\mathcal{Y}, m, z_1, z_2))$ in the list $\mathcal{L}_1$.

Oracle $\mathcal{O}^S(\mathcal{Y}, m)$: Upon receiving a message m and a public key list $\mathcal{Y} \subseteq \mathcal{PK}$ from $\mathcal{A}$, $\mathcal{C}$ first acquires $(\tilde{Y}_1 = [r]Y_1, \tilde{Y}_2 = [r]Y_2)$ by querying $\mathcal{O}^{H_1}$ with $\mathcal{Y}$ and randomly selects $\pi \in \{1, \dots, \ell\}$, where ℓ is the size of $\mathcal{Y}$. Then, $\mathcal{C}$ computes $\tilde{pk} = r \cdot pk_\pi$, implicitly indicating that $\mathcal{C}$ attempts to output a signature σ signed with the secret key sk_π . For $i \in \{1, \dots, \ell\}$, $\mathcal{C}$ computes $z_{i1} = [\beta_i]Y_1 + v_i \tilde{pk} + [\beta'_i]Y_2$ and $z_{i2} = [\beta_i]\tilde{Y}_1 + v_i \tilde{pk} + [\beta'_i]\tilde{Y}_2$, where $c_i, v_i, v_{i+1}, \beta_i, \beta'_i \leftarrow_{\$} \mathbb{Z}_q^*$. Subsequently, $\mathcal{C}$ sets $H_2(\mathcal{Y}, m, z_{i1}, z_{i2}) = v_{i+1}$ and records $(\mathcal{Y}, m, z_i, H_2(\mathcal{Y}, m, z_{i1}, z_{i2}))$ in the list $\mathcal{L}_0$. Finally, $\mathcal{C}$ outputs the signature $\sigma = (v_1, \beta_1, \dots, \beta_\ell, \beta'_1, \dots, \beta'_\ell, \tilde{pk})$ to complete this query.

- **Challenge:** The adversary $\mathcal{A}$ selects $pk_0^*, pk_1^* \leftarrow_{\$} \mathcal{PK}$ and a message m^* . Then, $\mathcal{A}$ sets the public key list $\mathcal{Y}^* = \{pk_0^*, pk_1^*\}$ and sends $(\mathcal{Y}^*, m^*)$ to the challenger $\mathcal{C}$. Implicitly, the corresponding secret keys of pk_0^* and

pk_1^* are $sk_0^* = (sk_{00} = ac_{00}, sk_{01} = ac'_{01})$ and $sk_1^* = (sk_{10} = ac_{10}, sk_{11} = ac'_{11})$, respectively. Subsequently, the challenger $\mathcal{C}$ queries the oracle $\mathcal{O}^{\text{H}_1}$ with $\mathcal{Y}^*$ to obtain the output $(\tilde{Y}_1 = H_1(\mathcal{Y}^*, 1), \tilde{Y}_2 = H_1(\mathcal{Y}^*, 2))$, where $H_1(\mathcal{Y}^*, 1) = [b]Y_1$, $H_1(\mathcal{Y}^*, 2) = [r] \cdot [b]Y_1$, $r \leftarrow_{\$} \mathbb{Z}_q^*$, and $(\mathcal{Y}^*, r, H_1(\mathcal{Y}^*, 1), H_1(\mathcal{Y}^*, 2))$ is recorded in the list $\mathcal{L}_0$. The challenger $\mathcal{C}$ then chooses $k \leftarrow_{\$} \{0, 1\}$ and computes $\tilde{\text{pk}} = [c_{0k}]T + [r \cdot c'_{1k}]T = [c_{0k} + r \cdot c'_{1k}]T$. To generate a valid signature σ^* for m^* and $\mathcal{Y}^*$, for $i \in \{1, 2\}$, $\mathcal{C}$ selects random elements $\{c_i^*, v_i^*, \beta_i^*, \beta_i'^*\}$ from $\mathbb{Z}_q^*$ and computes $z_{i1}^* = [\beta_i^*]Y_1 + v_i^* \tilde{\text{pk}} + [\beta_i'^*]Y_2$ and $z_{i2}^* = [\beta_i^*]\tilde{Y}_1 + v_i^* \tilde{\text{pk}} + [\beta_i'^*]\tilde{Y}_2$. Moreover, $\mathcal{C}$ sets $H_2(\mathcal{Y}^*, m^*, z_{i1}^*, z_{i2}^*) = v_{i+1}^*$ and records it in the list $\mathcal{L}_1$. Finally, $\mathcal{C}$ outputs the signature $\sigma^* = (v_1^*, \beta_1^*, \beta_2^*, \beta_1'^*, \beta_2'^*, \tilde{\text{pk}})$.

- **Phase 2:** The adversary proceeds as in Phase 1
- **Guess:** The adversary outputs a guess k' on k . If $k' = k$, we say that the adversary wins the game.

If $T = [a \cdot b]Y_1$, then we have a valid signature σ due to $\tilde{\text{pk}} = [c_{0k}]T + [r \cdot c'_{1k}]T = [c_{0k} \cdot a \cdot b]Y_1 + [r \cdot c'_{1k} \cdot a \cdot b]Y_1 = [a \cdot c_{0k} \cdot b]Y_1 + [a \cdot c'_{1k} \cdot b \cdot r]Y_1 = [a \cdot c_{0k}]\tilde{Y}_1 + [a \cdot c'_{1k}]\tilde{Y}_2$. In this case, the probability $b = b'$ equals the probability that the adversary $\mathcal{A}$ breaks the signer ambiguity. If T is a random value, then the probability of $b = b'$ equals $1/2$.

Hence, we obtain this theorem. $\square$

III. SECURITY ANALYSIS OF DBZKRA

1) *Security Model and Analysis of DBZKRA:* Our DBZKRA scheme satisfies completeness, computational soundness, and zero-knowledge. The specific illustrations are as follows:

- **Completeness.** For $pp \leftarrow \text{Setup}(1^\lambda)$, any statement/witness pair (s, w) satisfying the relation $\text{Rel}(s, w) = 1$, the equation $1 = \text{Verify}(s, \text{Prove}(s, w))$ must hold with probability 1. The formal description is given as follows:

$$\Pr \left[\begin{array}{l} \text{Verify}(s, \pi) = 1 : \\ \text{Rel}(s, w) = 1 \end{array} : \begin{array}{l} pp \leftarrow \text{Setup}(1^\lambda) \\ \pi \leftarrow \text{Prove}(s, w) \end{array} \right] = 1.$$

- **Computational soundness.** For $pp \leftarrow \text{Setup}(1^\lambda)$, the probability that any PPT adversary $\mathcal{A}$ outputs a statement/witness pair (s, w) that does not satisfy the relation $\text{Rel}(s, w) = 1$, but for which the equation $1 = \text{Verify}(s, \text{Prove}(s, w))$ holds, is always negl . The formal description is given as follows:

$$\Pr \left[\begin{array}{l} \text{Verify}(s, \pi) = 1 \\ \wedge \text{Rel}(s, w) \neq 1 \end{array} : \begin{array}{l} pp \leftarrow \text{Setup}(1^\lambda) \\ (s, w) \leftarrow \mathcal{A}(pp) \\ \pi \leftarrow \text{Prove}(s, w) \end{array} \right] \leq \text{negl}.$$

- **Perfect Special Honest-Verifier Zero-Knowledge.** A public coin argument of knowledge (Setup , Prove , Verify) is a perfect special honest verifier zero knowledge (SHVZK) argument of knowledge for Rel if there

exists a PPT simulator $\mathcal{S}$, such that for any pair of interactive adversaries $\mathcal{A}_1, \mathcal{A}_2$,

$$\Pr \left[\begin{array}{l} (s, w) \in \text{Rel} \\ \wedge \mathcal{A}_1(\text{tr}) = 1 \end{array} : \begin{array}{l} pp \leftarrow \text{Setup}(1^\lambda) \\ (s, w, \rho) \leftarrow \mathcal{A}_2(pp) \\ \text{tr} \leftarrow (\text{Prove}(s, w), \\ \text{Verify}(s, \pi)) \end{array} \right] \\ = \Pr \left[\begin{array}{l} (s, w) \in \text{Rel} \\ \wedge \mathcal{A}_1(\text{tr}) = 1 \end{array} : \begin{array}{l} pp \leftarrow \text{Setup}(1^\lambda) \\ (s, w, \rho) \leftarrow \mathcal{A}_2(pp) \\ \text{tr} \leftarrow \mathcal{S}(s, \rho) \end{array} \right],$$

where ρ is the public coin randomness used by the verifier.

We resort to the Universal Composability (UC) framework [2] to define the security of DBZKRA. The inherent property of UC ensures that a scheme proven secure under this framework continues to meet its security goals, even when executed concurrently with other schemes or when multiple instances run simultaneously.

In the UC framework, the security model is structured around two paradigms: the real world and the ideal world. In the real world, both the parties and the adversary $\mathcal{A}$ participate in executing the protocol Π . The parties receive input parameters from the environment $\mathcal{E}$ and return the outputs generated by Π , while the adversary $\mathcal{A}$ attempts to disrupt the protocol Π and outputs the results generated by Π . Conversely, in the ideal world, the functionality $\mathcal{F}$ acts as a trusted third party. In this setting, a simulator $\mathcal{S}$ emulates the behavior of the adversary $\mathcal{A}$ to interact with $\mathcal{F}$, returning the output. The parties communicate directly with $\mathcal{F}$, using inputs provided by the environment $\mathcal{E}$ and returning the corresponding outputs from $\mathcal{F}$. The output of the environment $\mathcal{E}$ during its interaction with the adversary $\mathcal{A}$ and the protocol Π in the real world is denoted as $\text{EXEC}_{\Pi, \mathcal{E}, \mathcal{A}}(1^\lambda)$. Similarly, in the ideal world, the output of the environment $\mathcal{E}$ when interacting with the simulator $\mathcal{S}$ and the ideal functionality $\mathcal{F}$ is denoted as $\text{EXEC}_{\mathcal{F}, \mathcal{E}, \mathcal{S}}(1^\lambda)$. Security is formally defined based on the indistinguishability of these two executions.

Definition 1. A protocol Π operating in the real world is said to UC-realize the ideal functionality $\mathcal{F}$ if, for any PPT adversary $\mathcal{A}$, there exists a simulator $\mathcal{S}$ such that

$$\text{EXEC}_{\Pi, \mathcal{E}, \mathcal{A}}(1^\lambda) \approx \text{EXEC}_{\mathcal{F}, \mathcal{E}, \mathcal{S}}(1^\lambda),$$

where $\approx$ indicates that the two distributions are computationally indistinguishable.

Our ideal functionality $\mathcal{F}_{\text{DBZKRA}}$ is based on the ideal functionality $\mathcal{F}_{\text{DBRSign}}$, which includes two interfaces: $\text{Sign}_{\text{DBRSign}}(m, \mathcal{Y}, sk)$ and $\text{Verify}_{\text{DBRSign}}(\sigma, m, \mathcal{Y})$. Specifically, the interface $\text{Sign}_{\text{DBRSign}}(m, \mathcal{Y}, sk)$ is used to generate a signature σ for the message m and the public key list $\mathcal{Y}$ with the secret key sk , while the interface $\text{Verify}_{\text{DBRSign}}(\sigma, m, \mathcal{Y})$ is used to check the validity of σ . As shown below, the defined ideal functionality $\mathcal{F}_{\text{DBZKRA}}$ consists of two interfaces: $\text{Prove}_{\text{DBZKRA}}(s, w)$ and $\text{Verify}_{\text{DBZKRA}}(s, \pi)$, where $s = [x]G_1 + [r_1]Y_1 + [r_2]Y_2$ is the statement, $w = (x, r_1, r_2)$ is the witness, and π is the proof. Notably, the prover executes the interface $\text{Prove}_{\text{DBZKRA}}(s, w)$, while

the verifier invokes the interface $\text{Verify}_{\text{DBZKRA}}(s, \pi)$ to check the validity of π outputted by the prover. Furthermore, we set the maximum bit length of the number hidden in s to be N , namely $x \in [0, 2^N - 1]$.

Ideal Functionality $\mathcal{F}_{\text{DBZKRA}}$	
$\text{Prove}(s, w)$	// $s = [x]G_1 + [r_1]Y_1 + [r_2]Y_2, w = (x, r_1, r_2)$
Upon invocation by the prover who proves $x \in [0, 2^N - 1]$:	
- convert x into its binary for: $x = \sum_{i=0}^N b_i 2^i$	
// b_i is the i -th binary digit of x with $b_i \in \{0, 1\}$	
- selects $N + 1$ random elements r_{1i} satisfying $r_1 = \sum_{i=0}^N r_{1i}$	
- selects $N + 1$ random elements r_{2i} satisfying $r_2 = \sum_{i=0}^N r_{2i}$	
- computes $s_i = [b_i 2^i]G_1 + [r_{1i}]Y_1 + [r_{2i}]Y_2$ for each b_i	
- computes the public key list $\mathcal{Y}_i = \{pk_{0i} = (s_i - [2^i]G_1), pk_{1i} = s_i\}$	
- generate σ_i by invoking $\text{Sign}_{\text{DBRSig}}(m, \mathcal{Y}_i, sk_i = (r_{1i}, r_{2i}))$	
// m could be any random element	
- output the proof $\pi = \{m, \sigma_0, \sigma_1, \dots, \sigma_N, s_0, s_1, \dots, s_N\}$	
$\text{Verify}(s, \pi)$	
Upon invocation by the verifier who verifies s and π :	
- return 0 if $s \neq \prod_{i=0}^N s_i$	
- computes the public key list $\mathcal{Y}_i = \{pk_{0i} = (s_i - [2^i]G_1), pk_{1i} = s_i\}$	
- return 0 if $\text{Verify}_{\text{DBZKRA}}(\sigma_i, m, \mathcal{Y}_i) = 0$ for each $i \in [0, \dots, N]$	
- return 1	

It is evident that our ideal functionality $\mathcal{F}_{\text{DBZKRA}}$ achieves the following security properties.

- **Completeness.** For the statement $s = [x]G_1 + [r_1]Y_1 + [r_2]Y_2$ and the witness $w = (x, r_1, r_2)$, it is evident that the following equation holds: $s = [x]G_1 + [r_1]Y_1 + [r_2]Y_2 = \sum_{i=0}^N s_i = \sum_{i=0}^N [b_i \cdot 2^i]G_1 + [r_{1i}]Y_1 + [r_{2i}]Y_2$, where $x = \sum_{i=0}^N b_i 2^i$, $r_1 = \sum_{i=0}^N r_{1i}$, and $r_2 = \sum_{i=0}^N r_{2i}$. Meanwhile, the prover always possesses a secret key $sk_i = (r_{1i}, r_{2i})$ corresponding to a public key in the public key list $\mathcal{Y}_i$. Consequently, the corresponding σ_i , generated by the interface $\text{Sign}_{\text{DBRSig}}(m, \mathcal{Y}_i, sk_i)$, can always be successfully verified.
- **Computational Soundness.** When the prove does not holds the statement $w = (x, r_1, r_2)$ satisfying $s = [x]G_1 + [r_1]Y_1 + [r_2]Y_2$, he/she could not generate valid signatures $\{\sigma_i\}_{i=0}^N$ served as the proof π .
- **Special Honest-Verifier Zero-Knowledge.** Since the secret value x is hidden within the statement s , and every bit value b_i of x is embedded into a public key in the public key list $\mathcal{Y}_i$, the verifier learns nothing about x except whether $x \in [0, 2^N - 1]$.

2) **Security Analysis of DBZKRA:** We formally prove that our DBZKRA satisfies completeness, computational Soundness,

and special honest-verifier zero-knowledge within the UC framework.

Theorem 6. *The DBZKRA scheme UC-realizes the ideal functionality $\mathcal{F}_{\text{DBZKRA}}$ in the $\mathcal{F}_{\text{DBRSig}}$ -hybrid framework.*

Proof. We define the following hybrid sequences transitioning from the original DBZKRA scheme to the ideal functionality $\mathcal{F}_{\text{DBZKRA}}$. Specifically, we incrementally modify the initial game and prove the indistinguishability of adjacent games.

$\mathcal{G}_0$: It is identical to the DBZKRA described in Section ??.

$\mathcal{G}_1$: It is almost the same as $\mathcal{G}_0$ except for the following condition: $\mathcal{G}_1$ aborts if the adversary $\mathcal{A}$ uses the interface $\text{Prove}(s, w)$ to generate a proof π , but the interface $\text{Verify}(s, \pi)$ returns 0.

$\mathcal{G}_2$: It is almost the same as $\mathcal{G}_1$, except for the following condition: $\mathcal{G}_2$ aborts if the adversary $\mathcal{A}$ outputs a proof π satisfying $\text{Verify}(s, \pi) = 1$ but without knowing the corresponding witness w for the statement s .

$\mathcal{G}_3$: It is almost the same as $\mathcal{G}_2$, except for the following condition: $\mathcal{G}_3$ aborts if the adversary $\mathcal{A}$ computes information other than whether $\text{Rel}(s, w) = 1$ given (s, π) . $\square$

Next, we will prove the adjacent games are indistinguishable in the environment $\mathcal{E}$.

Lemma 1. *For all PPT distinguisher $\mathcal{E}$ it holds that $\text{EXEC}_{\mathcal{G}_0, \mathcal{E}, \mathcal{A}}(1^\lambda) \equiv \text{EXEC}_{\mathcal{G}_1, \mathcal{E}, \mathcal{A}}(1^\lambda)$.*

Proof. According to the description of DBZKRA, for the statement $s = [x]G_1 + [r_1]Y_1 + [r_2]Y_2$, the secret value x is converted into its binary format as $x = \sum_{i=0}^N b_i 2^i$. Moreover, the chosen random elements $\{r_{1i}\}_{i=0}^N$ and $\{r_{2i}\}_{i=0}^N$ must satisfy the conditions $r_1 = \sum_{i=0}^N r_{1i}$ and $r_2 = \sum_{i=0}^N r_{2i}$. In this case, we set $s_i = [2^i b_i]G_1 + [r_{1i}]Y_1 + [r_{2i}]Y_2$ ($i \in \{0, \dots, N\}$), leading to the equation $s = \sum_{i=0}^N s_i$. Regarding each ring signature σ_i in the proof π , it is correctly generated by invoking the algorithm $\text{DBRSig.Sign}(m, \mathcal{Y}_i, sk_i)$, where $\mathcal{Y}_i = \{pk_0 = s_i, pk_1 = s_i - [2^i]G_1\}$ and $sk_i = (r_{1i}, r_{2i})$. As a result, the correctness of the scheme DBRSig ensures that each ring signature σ_i successfully passes the check in the algorithm DBZKRA.Verify .

Therefore, $\mathcal{G}_0$ and $\mathcal{G}_1$ are indistinguishable. $\square$

Lemma 2. *For all PPT distinguisher $\mathcal{E}$ it holds that $\text{EXEC}_{\mathcal{G}_1, \mathcal{E}, \mathcal{A}}(1^\lambda) \approx \text{EXEC}_{\mathcal{G}_2, \mathcal{E}, \mathcal{A}}(1^\lambda)$.*

Proof. When $\mathcal{G}_1$ and $\mathcal{G}_2$ are distinguishable, it indicates that the adversary $\mathcal{A}$ has generated a valid proof π without knowing the correct witness (x, r_1, r_2) satisfying $s = [x]G_1 + [r_1]Y_1 + [r_2]Y_2$. Furthermore, the securely proven unforgeability of the scheme DBRSig in Appendix II-B guarantees that the probability of any PPT adversary $\mathcal{A}$ generating a valid signature without the secret key is negligible. In this case, the only way for the adversary $\mathcal{A}$ to generate a valid proof π is to output (x^*, r_1^*, r_2^*) satisfying $s = [x^*]G_1 + [r_1^*]Y_1 + [r_2^*]Y_2$.

Below, we show that the adversary $\mathcal{A}$ producing such a tuple satisfying the above condition is equivalent to solving DL problem given $(G_1, [a]G_1)$. Specifically, we set $Y_1 = [\alpha]G_1$ and $Y_2 = [\beta]G_2$, and compute $s = [a]G_1 + [r_1]Y_1 + [r_2]Y_2$, where $\alpha, \beta, r_1, r_2 \leftarrow_{\$} \mathbb{Z}_q^*$. If the adversary $\mathcal{A}$ outputs

(x^*, r_1^*, r_2^*) satisfying $s = [x^*]G_1 + [r_1^*]Y_1 + [r_2^*]Y_2$ with non-negligible probability, then we can derive the DL solution a with the same probability based on the following equations:

$$\begin{aligned}
s &= [a]G_1 + [r_1]Y_1 + [r_2]Y_2 \\
&= [a]G_1 + [r_1 \cdot \alpha]G_1 + [r_2 \cdot \beta]G_1 \\
&= [a + r_1 \cdot \alpha + r_2 \cdot \beta]G_1 \\
&= [x^*]G_1 + [r_1^* \cdot \alpha]G_1 + [r_2^* \cdot \beta]G_1 \\
&= [x^*]G_1 + [r_1^* \cdot \alpha]G_1 + [r_2^* \cdot \beta]G_1 \\
&= [x^* + r_1^* \cdot \alpha + r_2^* \cdot \beta]G_1, \\
a &= x^* + r_1^* \cdot \alpha + r_2^* \cdot \beta - r_1 \cdot \alpha + r_2 \cdot \beta.
\end{aligned}$$

Therefore, $\mathcal{G}_1$ and $\mathcal{G}_2$ are indistinguishable. $\square$

Lemma 3. For all PPT distinguisher $\mathcal{E}$ it holds that $EXEC_{\mathcal{G}_2, \mathcal{E}, \mathcal{A}}(1^\lambda) \approx EXEC_{\mathcal{G}_3, \mathcal{E}, \mathcal{A}}(1^\lambda)$.

Proof. When $\mathcal{G}_2$ and $\mathcal{G}_3$ are distinguishable, it indicates that the adversary $\mathcal{A}$ has extracted information beyond the event $\text{Rel}(s, w) = 1$. However, our proposed DBZKRA scheme is constructed based on the ring signature DBRSign, which has been thoroughly proven to ensure anonymity in Appendix III-2. In other words, the adversary $\mathcal{A}$ cannot identify the signer (bit value) in the public key list $\mathcal{Y}_i$ given $(\mathcal{Y}_i, \sigma_i)$. Furthermore, for the given statement $s = [x]G + [r_1]Y_1 + [r_2]Y_2$, there exist numerous triples (x^*, r_1^*, r_2^*) from $\mathbb{Z}_q^*$ that satisfy $s = [x^*]G + [r_1^*]Y_1 + [r_2^*]Y_2$. These triples are indistinguishable to the adversary $\mathcal{A}$, even if it possesses unlimited computational resources.

Therefore, $\mathcal{G}_2$ and $\mathcal{G}_3$ are indistinguishable. $\square$

Lemma 4. For all PPT distinguisher $\mathcal{E}$ it holds that $EXEC_{\mathcal{G}_3, \mathcal{E}, \mathcal{A}}(1^\lambda) = EXEC_{\mathcal{F}_{\text{DBZKRA}}, \mathcal{E}, \mathcal{A}}(1^\lambda)$.

Proof. The differences between $\mathcal{G}_3$ and the ideal functionality $\mathcal{F}_{\text{DBZKRA}}$ are only conceptual.

Hence, $\mathcal{G}_4$ and $\mathcal{F}_{\text{DBZKRA}}$ are indistinguishable. $\square$

This concludes the proof of Theorem 6.

IV. SECURITY MODEL AND ANALYSIS OF EVA²Hub

A. Security Model of EVA²Hub

In this section, we formally define EVA²Hub and outline its corresponding security within the UC framework [2]. For simplicity, we use the notations A, H, and B to denote the payer, the tumbler, and the payee in the payment channel, respectively. Following the guidelines in [3], [4], we model the security of EVA²Hub within the UC framework [2], specifically under the Global UC (GUC) framework [5]. In the GUC framework, based on seven foundational functionalities, $\mathcal{F}_{\text{clock}}$, $\mathcal{F}_{\text{smt}}$, $\mathcal{F}_{\text{anon}}$, $\mathcal{F}_{\text{DBSign}}$, $\mathcal{F}_{\text{DBZKRA}}$, and $\mathcal{F}_{\text{BC}}$, we define our ideal functionality $\mathcal{F}_{\text{EVA}^2\text{Hub}}$ as illustrated below. Specifically, $\mathcal{F}_{\text{clock}}$ is a global ideal functionality ensuring that all honest parties have completed their communication for the current round and are prepared for the next. $\mathcal{F}_{\text{smt}}$ provides secure communication, ensuring that messages sent by users cannot be intercepted or altered by adversaries; the interface send_{smt} is used for message transmission via $\mathcal{F}_{\text{smt}}$. $\mathcal{F}_{\text{anon}}$ represents an anonymous communication channel that prevents adversaries

from identifying the payer of a message, with the interface $\text{send}_{\text{anon}}$ enabling message transmission through $\mathcal{F}_{\text{anon}}$.

Moreover, we define $\mathcal{F}_{\text{DBSign}}$, $\mathcal{F}_{\text{DBPCom}}$, and $\mathcal{F}_{\text{DBZKRA}}$ as the ideal functionalities corresponding to the schemes DBSign, DBPCom, and DBZKRA, respectively. Specifically, the ideal functionality $\mathcal{F}_{\text{DBSign}}$ includes four interfaces. The $\text{BldMsg}_{\text{DBSign}}(x)$ interface generates the first- and second-level blinded messages (m_b, m_{bb}) along with (r_1, r_2) . The $\text{Sig}_{\text{DBSign}}(sk, m_{bb})$ interface generates a valid second-level blinded signature σ_{bb} using the secret key sk , and the $\text{UblidSig}_{\text{DBSign}}(\sigma_{bb}, m_{bb}, m_b, r_2)$ interface unblinds the second-level blinded signature σ_{bb} into the first-level blinded signature σ_b . The $\text{VrfSecBldMsg}_{\text{DBSign}}(x, r_1, r_2, m_{bb})$ interface verifies whether the original message x is correctly blinded into m_{bb} using the random factors r_1 and r_2 . Finally, the validity of both first- and second-level blinded messages and signatures for pk can be verified by invoking the $\text{Verify}_{\text{DBSign}}(pk, m_k, \sigma_k)$ interface, where $k \in \{b, bb\}$. For the ideal functionality $\mathcal{F}_{\text{DBZKRA}}$, it provides two interfaces: $\text{Prove}_{\text{DBZKRA}}(s, w)$ and $\text{Verify}_{\text{DBZKRA}}(s, \pi)$. Each interface takes the same inputs and produces the same outputs as the corresponding algorithm with the same name in the scheme DBZKRA.

Regarding $\mathcal{F}_{\text{BC}}$, we model the functionality of the underlying blind channel (BC), which provides interfaces for operating a blind channel in the ideal world. It is worth noting that one party of each blind channel is fixed as the tumbler H, due to the star topology of the PCH. Furthermore, with the introduction of the concept of blinded balances, $\mathcal{F}_{\text{BC}}$ uses bbl to denote such values.

Ideal Functionality $\mathcal{F}_{\text{PC}}$

OPEN. Upon receiving $(\mathcal{U}, H, \text{bal}_{\mathcal{U}}, \text{bal}_H, \text{bbl}_{\mathcal{U}}, \text{bbl}_H, \text{AuInfo})$, parse $\text{AuInfo} = (r_{\mathcal{U}1}, r_{\mathcal{U}2}, r_{H1}, r_{H2})$ and use it to verify the correctness of the blinded balances $\text{bbl}_{\mathcal{U}}$ and bbl_H . If the verification succeeds, initialize the blind channel as $\tilde{\zeta} = \{\text{id}, \text{state} = 0, \text{parties} = (\mathcal{U}, H), \text{bal}_{\mathcal{U}}, \text{bal}_H, \text{bbl}_{\mathcal{U}}, \text{bbl}_H\}$, where id denotes the channel identifier. The functionality then records $\tilde{\zeta}$ on the blockchain and returns it.

UPDATE. Upon receiving $(\zeta, \text{bbl}_{\mathcal{U}}, \text{bbl}_H, \text{prf})$ from the tumbler H, or $(\zeta, \text{bal}_{\mathcal{U}}, \text{bal}_H, \text{bbl}_{\mathcal{U}}, \text{bbl}_H, \text{prf})$ from the user $\mathcal{U}$, the functionality first verifies the validity of the proof prf for the update. If valid, it updates $(\text{bbl}_{\mathcal{U}}, \text{bbl}_H)$ accordingly. Additionally, $(\text{bal}_{\mathcal{U}}, \text{bal}_H)$ can be updated only if the user invokes the interface with a valid proof prf . Finally, the channel state is incremented by setting $\zeta.\text{state} \leftarrow \zeta.\text{state} + 1$, and the updated ζ is returned.

CLOSE. Upon receiving (ζ, prf) , the functionality checks whether prf is valid for closing the channel based on ζ . If the check passes, the channel state is published on the blockchain, and a timeout period of 2Δ is returned. If the counterparty submits a newer channel state ζ' such that $\zeta'.\text{state} > \zeta.\text{state}$ with a valid

proof prf within 2Δ , then the original invoker forfeits all funds in the channel. Otherwise, the functionality will automatically require the user to disclose the real balances associated with ζ via the PUBLISH interface.

PUBLISH. Upon receiving $(\text{bal}_U, \text{bal}_H, \text{prf})$ from the user U , the functionality checks whether prf is valid for the channel ζ recorded in the CLOSE interface. If valid, it finalizes the channel and returns the corresponding balances to the respective accounts. Otherwise, all funds in the channel are assigned to the tumbler H.

As shown in below, our ideal functionality $\mathcal{F}_{\text{EVA}^2\text{Hub}}$ consists of three procedures: (A) OPEN, (B) PAYMENT, and (C) CLOSE. In the OPEN procedure, the user (either Alice A or Bob B) cooperatively establishes a payment channel with the tumbler H by uploading the corresponding transaction to the blockchain. Following this, both parties convert the payment channel into a blind channel, which forms the foundation for the off-chain privacy-preserving transactions conducted in the subsequent PAYMENT procedure. In this procedure, Alice A securely transfers coins to Bob B through the tumbler H, ensuring privacy by invoking the aforementioned interfaces. Finally, in the CLOSE procedure, the parties retrieve their balances from the channel and transfer them back to the blockchain.

Ideal Functionality $\mathcal{F}_{\text{EVA}^2\text{Hub}}$

(A) Open

1. The user U (i.e., A or B) and the hub H collaboratively construct an on-chain transaction to open a payment channel, and publish it to the ledger $\mathcal{L}(\Delta)$. Once confirmed, the channel between U and H is considered established.

2. Next, both parties blind their initial channel balances. For example, with Alice A's channel defined as $\zeta = \{\text{id}, \text{state} = 0, \text{parties} = \{A, H\}, \text{bal}_A, \text{bal}_H\}$, she runs $\text{FirBldMsgDBSign}(\text{bal}_A)$ to obtain $(m_{Ab}, \zeta.r_{A1})$, and then $\text{SecBldMsgDBSign}(m_{Ab})$ to derive $(\zeta.\text{bbal}_A, \zeta.r_{A2})$. Finally, Alice sends $(\zeta.\text{bbal}_A, \zeta.r_{A1}, \zeta.r_{A2})$ to H via send_{smt} .

3. Upon receiving the blinded balance, the hub H verifies its correctness using $\text{VrfSecBldMsgDBSign}(\zeta.\text{bal}_A, \zeta.r_{A1}, \zeta.r_{A2}, \zeta.\text{bbal}_A)$. If verification fails, the protocol aborts. Otherwise, both parties invoke the ideal functionality $\mathcal{F}_{\text{BC}}.\text{OPEN}(U, H, \text{bal}_U, \text{bal}_H, \text{bbal}_U, \text{bbal}_H, \text{AuInfo})$ to finalize the channel initialization. We denote the resulting blind channels as ζ (between A and H) and $\tilde{\zeta}$ (between B and H).

(B) Payment

// Blind Transfer from Alice to Bob via the hub.

1. Alice A first computes $(m_{bb}, m_b, r_1, r_2) \leftarrow \text{BldMsg}_{\text{DBSign}}(x)$. She then generates $\pi_1 \leftarrow \text{Prove}_{\text{DBZKRA}}(m_{bb}, (x, r_1, r_2))$ to prove $x \geq 0$. To prove $\zeta.\text{bal}_A \geq x + \text{fee}$, she computes: $\text{bal}'_A = \zeta.\text{bal}_A - x - \text{fee}$, $\text{bbal}'_A = \zeta.\text{bbal}_A - m_{bb} - [\text{fee}]G_1$,

and $r'_{Ai} = \zeta.r_{Ai} - r_i$ for $i = 1, 2$, then obtains $\pi_2 \leftarrow \text{Prove}_{\text{DBZKRA}}(\text{bbal}'_A, (r'_{A1}, r'_{A2}))$. She sends (m_{bb}, π_1, π_2) to the hub H.

2. The hub H verifies π_1 and π_2 , and if valid, signs m_{bb} as $\sigma_{bb} \leftarrow \text{Sign}_{\text{DBSign}}(\text{sk}, m_{bb})$, updates $\text{bbal}_A \leftarrow m_{bb} + [\text{fee}]G_1$ and $\text{bbal}_H \leftarrow m_{bb} + [\text{fee}]G_1$, and calls $\mathcal{F}_{\text{BC}}.\text{Update}(\zeta, \text{bbal}_A, \text{bbal}_H)$. Then, it sends σ_{bb} to Alice.

3. Alice A verifies σ_{bb} via $\text{Verify}_{\text{DBSign}}(\text{pk}, m_{bb}, \sigma_{bb})$. If valid, she updates: $\zeta.\text{bal}_A \leftarrow x + \text{fee}$, $\zeta.\text{bal}_H \leftarrow x + \text{fee}$; $\zeta.r_{Ai} \leftarrow r_i$, $\zeta.r_{Hi} \leftarrow r_i$ for $i = 1, 2$, and adjusts $(\zeta.\text{bbal}_A, \zeta.\text{bbal}_H)$ accordingly. Alice then invokes $\mathcal{F}_{\text{BC}}.\text{Update}(\zeta, \zeta.\text{bal}_A, \zeta.\text{bal}_H, \zeta.\text{bbal}_A, \zeta.\text{bbal}_H, \text{prf} = (m_{bb}, \pi))$ and sends $T = H(m_b)$ to Bob via the anonymous interface $\text{send}_{\text{anon}}$.

4. Bob B sends the token $T = H(m_b)$ along with $\tilde{\zeta}.\text{id}$ to the hub H to confirm its freshness. If T is unused, H replies with a signature $\sigma \leftarrow \text{Sign}_{\text{sk}}(T \parallel \tilde{\zeta}.\text{id})$ authorizing the upcoming payment. B then forwards $(\sigma, T, \tilde{\zeta}.\text{id})$ to Alice A via the anonymous interface $\text{send}_{\text{anon}}$, enabling her to compute $\sigma_b \leftarrow \text{UbldSig}_{\text{DBSign}}(\sigma_{bb}, m_{bb}, m_b, r_2)$ with the corresponding blinding factor r_1 . Upon completion, balances are updated as follows: $\tilde{\zeta}.\text{bal}_B \leftarrow x$, $\tilde{\zeta}.\text{bal}_H \leftarrow x$, $\tilde{\zeta}.\text{bbal}_B \leftarrow m_b$, $\tilde{\zeta}.\text{bbal}_H \leftarrow m_b$, $\tilde{\zeta}.r_{Bi} \leftarrow r_i$, $\tilde{\zeta}.r_{Hi} \leftarrow r_i$ for $i = 1, 2$. Bob then invokes $\mathcal{F}_{\text{BC}}.\text{Update}(\tilde{\zeta}, \tilde{\zeta}.\text{bal}_B, \tilde{\zeta}.\text{bal}_H, \tilde{\zeta}.\text{bbal}_B, \tilde{\zeta}.\text{bbal}_H)$ and sends σ_b to H. The hub verifies σ_b via $\text{DBSign}.\text{Verify}$ and, if valid, updates its local blinded balances: $\text{bbal}_B \leftarrow m_b$, $\text{bbal}_H \leftarrow m_b$, and calls $\mathcal{F}_{\text{BC}}.\text{Update}$ accordingly.

If T has already been used, H instead returns a signature $\sigma \leftarrow \text{Sign}_{\text{sk}}(T \parallel \text{id}^*)$, where id^* is the channel where T was consumed. Bob forwards (σ, T, id^*) to Alice via $\text{send}_{\text{anon}}$, who then triggers the CLOSE phase to resolve the token dispute through the on-chain contract.

(C) Close

The CLOSE phase handles both cooperative closure and dispute resolution.

Normal Closure. Consider channel $\zeta = (\text{id}, \text{state} = 10, \text{parties} = \{A, H\}, \text{bbal}_A, \text{bbal}_H)$. Either party may invoke $\mathcal{F}_{\text{BC}}.\text{Close}(\zeta, \text{prf})$, where prf contains signatures from both participants. If a stale state is submitted, the honest party may override it by submitting a fresher ζ with valid proofs within 2Δ . Once closure is initiated, A must reveal $(\zeta.\text{bal}_A, \zeta.\text{bal}_H, \zeta.r_{Ai}, \zeta.r_{Hi})$ for $i = 1, 2$ via $\mathcal{F}_{\text{BC}}.\text{Publish}$ within 2Δ .

Dispute Resolution. B starts dispute resolution by submitting (σ, T, id) to the smart contract. The contract verifies that σ is a valid signature on $T \parallel \text{id}$ under pk^Ψ and requires H to disclose m_b such that $T = H(m_b)$ within 2Δ . If H provides a valid m_b matching T , proving token reuse, it claims the full balance as penalty. Otherwise, A can claim all coins.

It is evident that our ideal functionality $\mathcal{F}_{\text{EVA}^2\text{Hub}}$ achieves the following security and privacy properties.

- *Atomicity*: This property is satisfied by $\mathcal{F}_{\text{EVA}^2\text{Hub}}$ as it ensures that only the tumbler H, possessing the secret key sk , can generate the valid σ_{bb} required to complete the sub-payment between Alice A and the tumbler H. Additionally, once Alice A forwards the valid m_{bb} to Bob B, B can derive a valid σ_b from σ_{bb} using the interface $\text{UbldSig}_{\text{DBSign}}$ to finalize the conditional payment between B and H.
- *Relationship Anonymity*: During the PAYMENT phase, the tumbler H has access only to (m_b, σ_b) and (m_{bb}, σ_{bb}) , where the latter pair is a randomized version of the former. Furthermore, Alice A and Bob B communicate through the anonymous communication interface $\text{send}_{\text{anon}}$. Consequently, $\mathcal{F}_{\text{EVA}^2\text{Hub}}$ fulfills this property.
- *Value Privacy*: The blind channels ζ and $\tilde{\zeta}$ in $\mathcal{F}_{\text{EVA}^2\text{Hub}}$ conceal the updated payment channel balances from the tumbler H. Additionally, the payment amount x is randomized through the interfaces $\text{FirBldMsg}_{\text{DBSign}}$ and $\text{SecBldMsg}_{\text{DBSign}}$. As a result, H is unable to discern the actual payment amount x .
- *Balance Security*: This property is satisfied by $\mathcal{F}_{\text{EVA}^2\text{Hub}}$ because Alice A must generate proofs (π_1, π_2) using the interface $\text{Prove}_{\text{DBZKRA}}$ to demonstrate the validity of the blind payment amount x . If the blind payment amount is invalid, such proofs cannot be successfully verified by the interface $\text{Verify}_{\text{DBZKRA}}$. Notably, $\mathcal{F}_{\text{EVA}^2\text{Hub}}$ does not require Bob B's to generate these proofs, as an invalid payment amount x would directly harm B's interests. Hence, $\mathcal{F}_{\text{EVA}^2\text{Hub}}$ ensures balance security.

B. Security Analysis of EVA^2Hub

We formally prove that our EVA^2Hub protocol satisfies atomicity, relationship anonymity, value privacy, variable amounts, and balance security within the UC framework.

Theorem 7. *The EVA^2Hub protocol UC-realizes the ideal functionality $\mathcal{F}_{\text{EVA}^2\text{Hub}}$ in the $(\mathcal{F}_{\text{clock}}, \mathcal{F}_{\text{smt}}, \mathcal{F}_{\text{anon}}, \mathcal{F}_{\text{DBSign}}, \mathcal{F}_{\text{DBZKRA}})$ -hybrid framework.*

Proof. We define the following hybrid sequences transitioning from the original EVA^2Hub protocol to the ideal functionality $\mathcal{F}_{\text{EVA}^2\text{Hub}}$. Specifically, we incrementally modify the initial game and prove the indistinguishability of adjacent games.

$\mathcal{G}_0$: It is identical to the EVA^2Hub described in Section ??.

$\mathcal{G}_1$: It is almost the same as $\mathcal{G}_0$ except for the following condition: $\mathcal{G}_0$ aborts if the adversary $\mathcal{A}$ outputs a valid proof $\pi = (\pi_1, \pi_2)$ such that $\text{DBZKRA.Verify}(m_{bb}, \pi_1) = 1$ and $\text{DBZKRA.Verify}(\text{bba}'_{\mathcal{A}}, \pi_2) = 1$ when $x < 0$ or $x + \text{fee} > \zeta.\text{bal}_{\mathcal{A}}$. Here, x is the payment amount hidden in m_{bb} , $\zeta.\text{bal}_{\mathcal{A}}$ and $\zeta.\text{bba}_{\mathcal{A}}$ are the adversary $\mathcal{A}$'s plaintext balance and blinded balance in ζ , respectively, and $\text{bba}'_{\mathcal{A}} = \zeta.\text{bba}_{\mathcal{A}} - m_{bb} - [\text{fee}]_{G_1}$.

$\mathcal{G}_2$: It is almost the same as $\mathcal{G}_1$ except for the following condition: $\mathcal{G}_2$ aborts if the adversary $\mathcal{A}$ without the private key sk outputs a valid second-level blinded signature $\sigma_{bb} = (\sigma_{bb}^{(0)}, \sigma_{bb}^{(1)}, \sigma_{bb}^{(2)})$ for m_{bb} such that $\hat{e}(\sigma_{bb}^{(0)}, G) = \hat{e}(G_2, \text{pk}) \cdot \hat{e}(m_{bb}, \sigma_{bb}^{(1)})$ and $\hat{e}(\sigma_{bb}^{(1)}, Y_2) = \hat{e}(G, \sigma_{bb}^{(2)})$.

$\mathcal{G}_3$: It is almost the same as $\mathcal{G}_2$ except for the following

condition: $\mathcal{G}_3$ aborts if the adversary $\mathcal{A}$, even with the private key sk , can determine the relation between (m_b, σ_b) and (m_{bb}, σ_{bb}) .

$\mathcal{G}_4$: It is almost the same as $\mathcal{G}_3$ except for the following condition: $\mathcal{G}_4$ aborts if the adversary $\mathcal{A}$ outputs the payment amount x hidden in m_b and m_{bb} . $\square$

Next, we prove the indistinguishability of adjacent games in the environment $\mathcal{E}$.

Lemma 5. *For all PPT distinguisher $\mathcal{E}$ it holds that $\text{EXEC}_{\mathcal{G}_0, \mathcal{E}, \mathcal{A}}(1^\lambda) \approx \text{EXEC}_{\mathcal{G}_1, \mathcal{E}, \mathcal{A}}(1^\lambda)$.*

Proof. Assuming that $\mathcal{G}_0$ and $\mathcal{G}_1$ are distinguishable, it implies that $\mathcal{G}_1$ aborts while $\mathcal{G}_0$ does not. In this scenario, the adversary $\mathcal{A}$ outputs a verified proof $\pi = (\pi_1, \pi_2)$, but either $x < 0$ or $x + \text{fee} > \zeta.\text{bal}_{\mathcal{A}}$. Consequently, we can leverage the adversary $\mathcal{A}$ to break the computational soundness of DBZKRA, which has been formally proved in Appendix III-2.

Hence, $\mathcal{G}_0$ and $\mathcal{G}_1$ are indistinguishable. $\square$

Lemma 6. *For all PPT distinguisher $\mathcal{E}$ it holds that $\text{EXEC}_{\mathcal{G}_1, \mathcal{E}, \mathcal{A}}(1^\lambda) \approx \text{EXEC}_{\mathcal{G}_2, \mathcal{E}, \mathcal{A}}(1^\lambda)$.*

Proof. Assuming that $\mathcal{G}_1$ and $\mathcal{G}_2$ are distinguishable, it implies that $\mathcal{G}_2$ aborts while $\mathcal{G}_1$ does not. In this scenario, the adversary $\mathcal{A}$, without possessing the private key sk , outputs a new valid second-level blinded signature σ_{bb} on m_{bb} . Consequently, we can leverage the adversary $\mathcal{A}$ to break the selective-existential unforgeability of DBSign, which has been formally proved in Appendix I-B.

Hence, $\mathcal{G}_1$ and $\mathcal{G}_2$ are indistinguishable. $\square$

Lemma 7. *For all PPT distinguisher $\mathcal{E}$ it holds that $\text{EXEC}_{\mathcal{G}_2, \mathcal{E}, \mathcal{A}}(1^\lambda) \approx \text{EXEC}_{\mathcal{G}_3, \mathcal{E}, \mathcal{A}}(1^\lambda)$.*

Proof. Assuming that $\mathcal{G}_2$ and $\mathcal{G}_3$ are distinguishable, it implies that $\mathcal{G}_3$ aborts while $\mathcal{G}_2$ does not. In this scenario, the adversary $\mathcal{A}$ could identify the relation between (m_b, σ_b) and (m_{bb}, σ_{bb}) . In other words, given two tuples $(m_{0b}, \sigma_{0b}, m_{0bb}, \sigma_{0bb})$ and $(m_{1b}, \sigma_{1b}, m_{1bb}, \sigma_{1bb})$, we randomly select $a \leftarrow_{\$} \{0, 1\}$ and provide $(m_{0b}, \sigma_{0b}, m_{1b}, \sigma_{1b}, m_{abb}, \sigma_{abb})$ to the adversary $\mathcal{A}$. The adversary $\mathcal{A}$ can correctly output the value of a . Consequently, we can leverage the adversary $\mathcal{A}$ to break second-level blindness of DBSign, which has been formally proved in Appendix I-B.

Hence, $\mathcal{G}_2$ and $\mathcal{G}_3$ are indistinguishable. $\square$

Lemma 8. *For all PPT distinguisher $\mathcal{E}$ it holds that $\text{EXEC}_{\mathcal{G}_3, \mathcal{E}, \mathcal{A}}(1^\lambda) \approx \text{EXEC}_{\mathcal{G}_4, \mathcal{E}, \mathcal{A}}(1^\lambda)$.*

Proof. Assuming that $\mathcal{G}_3$ and $\mathcal{G}_4$ are distinguishable, it implies that $\mathcal{G}_3$ aborts while $\mathcal{G}_2$ does not. In this scenario, the adversary $\mathcal{A}$ could output the payment amount x hidden in m_b and m_{bb} . Consequently, we can leverage the adversary $\mathcal{A}$ to break the first-level blindness of DBSign, which has been formally proved in Appendix I-B.

Hence, $\mathcal{G}_3$ and $\mathcal{G}_4$ are indistinguishable. $\square$

Lemma 9. *For all PPT distinguisher $\mathcal{E}$ it holds that $\text{EXEC}_{\mathcal{G}_4, \mathcal{E}, \mathcal{A}}(1^\lambda) = \text{EXEC}_{\mathcal{F}_{\text{EVA}^2\text{Hub}}, \mathcal{E}, \mathcal{A}}(1^\lambda)$.*

Proof. The differences between $\mathcal{G}_4$ and the ideal functionality $\mathcal{F}_{\text{EVA}^2\text{Hub}}$ are only conceptual.

Hence, $\mathcal{G}_4$ and $\mathcal{F}_{\text{EVA}^2\text{Hub}}$ are indistinguishable. $\square$

This concludes the proof of Theorem 7.

REFERENCES

- [1] J. K. Liu, V. K. Wei, and D. S. Wong, "Linkable spontaneous anonymous group signature for ad hoc groups (extended abstract)," in *ACISP*, ser. LNCS, vol. 3108. Springer, 2004, pp. 325–335.
- [2] R. Canetti, "Universally composable security: A new paradigm for cryptographic protocols," in *FOCS*. IEEE Computer Society, 2001, pp. 136–145.
- [3] Z. Ge, J. Gu, C. Wang, Y. Long, X. Xu, and D. Gu, "Accio: Variable-amount, optimized-unlinkable and nizk-free off-chain payments via hubs," in *ACM CCS*. ACM, 2023, pp. 1541–1555.
- [4] X. Qin, S. Pan, A. Mirzaei, Z. Sui, O. Ersoy, A. Sakzad, M. F. Esgin, J. K. Liu, J. Yu, and T. H. Yuen, "Blindhub: Bitcoin-compatible privacy-preserving payment channel hubs supporting variable amounts," in *IEEE S&P*. IEEE, 2023, pp. 2462–2480.
- [5] R. Canetti, Y. Dodis, R. Pass, and S. Walfish, "Universally composable security with global setup," in *TCC*, vol. 4392. Springer, 2007, pp. 61–85.

REFERENCES

- [1] J. K. Liu, V. K. Wei, and D. S. Wong, "Linkable spontaneous anonymous group signature for ad hoc groups (extended abstract)," in *ACISP*, ser. LNCS, vol. 3108. Springer, 2004, pp. 325–335.
- [2] R. Canetti, "Universally composable security: A new paradigm for cryptographic protocols," in *FOCS*. IEEE Computer Society, 2001, pp. 136–145.
- [3] Z. Ge, J. Gu, C. Wang, Y. Long, X. Xu, and D. Gu, "Accio: Variable-amount, optimized-unlinkable and nizk-free off-chain payments via hubs," in *ACM CCS*. ACM, 2023, pp. 1541–1555.
- [4] X. Qin, S. Pan, A. Mirzaei, Z. Sui, O. Ersoy, A. Sakzad, M. F. Esgin, J. K. Liu, J. Yu, and T. H. Yuen, "Blindhub: Bitcoin-compatible privacy-preserving payment channel hubs supporting variable amounts," in *IEEE S&P*. IEEE, 2023, pp. 2462–2480.
- [5] R. Canetti, Y. Dodis, R. Pass, and S. Walfish, "Universally composable security with global setup," in *TCC*, vol. 4392. Springer, 2007, pp. 61–85.



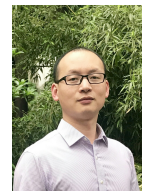
Shuyu Zheng is currently pursuing the master's degree with Zhejiang Gongshang University, China. Her current research interests include machine learning security and federated learning poisoning attack..



Cong Zuo (Member, IEEE) received the Ph.D. degree from Monash University, Australia, in 2020. From 2021 to 2022, he was a Research Fellow at Nanyang Technological University, Singapore. He is currently an Associate Professor at the School of Cyberspace Science and Technology, Beijing Institute of Technology (BIT), China. His main research interest is on cybersecurity and, in particular, database security and applied cryptography.



Xiaofeng Jia is a master student of the School of Computer Science and Technology at Zhejiang Gongshang University. His research interests include applied cryptography and blockchain.



Jun Shao (Senior Member, IEEE) received the Ph.D. degree from the Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai, China, in 2008. He is currently a Professor with the School of Computer Science and Technology, Zhejiang Gongshang University, Hangzhou, China. His current research interests include network security and applied cryptography.



Pubin Jiang is a master student of the School of Computer Science and Technology at Zhejiang Gongshang University. His research interests include applied cryptography and blockchain.